PLATEFORME DE TRAITEMENT DES PAIEMENTS

RESUME TECHNIQUE DES LIVRABLES

Nature du document	
Version	**1.0**
Client	**Étude de cas – STRIPE**
Date de la version	**16/10/2026**
Description de la version	**Bloc 2 - certification RNCP41993**
Modules concernés	**Livrables 0 à 9**
Auteur	**Ronan GUILLOU**

1 Résumé technique des livrables

Ce document résume, pour chaque livrable, ce qu'il décrit et les choix techniques retenus. Il sert de guide de lecture : le détail se trouve dans chaque livrable. La plateforme repose sur **trois systèmes de données** (Aurora PostgreSQL pour les paiements, un lakehouse Databricks pour l'analyse, DocumentDB pour les données non structurées), reliés par Kafka.

1.1 Où trouver quoi

Sujet	Où le trouver
Le besoin et le cadrage	L0, chapitres 1 et 2
Les 3 systèmes de données (paiements, analyse, NoSQL)	L1 (OLTP), L2 (OLAP), L3 (NoSQL)
La circulation des données	L4, chapitre 3 (flux) et chapitre 4 (orchestration)
La détection de fraude en moins de 100 ms	L3, chapitre 3 (compteurs Redis) ; L5, chapitre 2.2
L'intégration du machine learning	L5, chapitres 3 à 5 (variables, déploiement, suivi)
La boucle de retour de la banque (étiquettes de fraude)	L4, chapitre 3.4
La sécurité, les accès et le RGPD	L6, chapitres 3 à 5
La résilience et la reprise sur incident	L4, chapitre 5
L'architecture complète en un coup d'œil	L7 (20 composants)
Les requêtes SQL et NoSQL	L8

Fiche de spécifications fonctionnelles

Cadre le besoin de la plateforme de paiement Stripe et répartit les fonctionnalités entre les livrables.

- **Trois exigences :** fiabilité (aucun paiement perdu ni traité deux fois), rapidité (réponse immédiate au client) et sécurité (données de paiement et personnelles encadrées).
- **Cinq enjeux :** volume de paiements, analyses, données de natures différentes, systèmes dispersés, réglementations (RGPD, CCPA, PCI-DSS).
- **Fonctionnalités par domaine :** vérifier un paiement, l'enregistrer, faire circuler les données, les stocker selon leur nature, analyser l'activité, protéger et prouver la conformité, garantir la continuité.
- **Plan de réponse :** les livrables 1 à 8 couvrent chacun un ou plusieurs domaines.

ERD du système OLTP

Modèle relationnel normalisé du système transactionnel, hébergé dans Aurora PostgreSQL.

- **Trois schémas :** reference (devises, pays, catégories, produits), actor (marchands, clients, demandes RGPD) et payment (production).
- **Transaction en insertion seule :** une ligne par opération ; un remboursement est une nouvelle ligne liée au paiement d'origine. Partitionnement mensuel pour absorber le volume.
- **Idempotence :** table idempotency_key séparée, pour qu'une requête rejouée ne crée pas de second paiement.
- **Atomicité :** le paiement, son évaluation de fraude (score, décision, motifs, version du modèle) et l'événement outbox sont écrits dans une seule transaction.
- **Protection :** aucun numéro de carte (jeton, marque, quatre derniers chiffres, empreinte) ; données personnelles isolées dans customer ; audit_log sans valeurs copiées.
- **Dénormalisation ciblée :** taux de change, nom et prix du produit copiés sur la transaction, pour garder les analyses passées exactes.

ERD du système OLAP

Modèle en étoile du lakehouse (schéma gold), construit à partir de l'OLTP pour les analyses.

- **Trois tables de faits :** fact_transaction (un paiement ou un remboursement, refusés compris) et fact_payment_item (un produit d'un paiement), en insertion seule et fact_fraud pour le feedback de la banque.

- **Sens des montants :** montants de base positifs ; signed_amount_eur porte le signe, donc le net est un simple SUM.
- **Dimensions :** date, pays (rôle double : pays du paiement et pays de l'IP, indicateur de fraude), catégorie analytique, marchand, client, produit.
- **Fraude :** score et décision repris de fraud_assessment.
- **Pas de données personnelles :** ni e-mail ni nom dans les dimensions.
- **Hypothèse :** un remboursement porte sur la totalité du paiement, un seul par paiement.

Modèle de données NoSQL

Collections DocumentDB et compteurs Redis pour la détection de fraude.

- **Stratégies de relation :** embedding, référence, tableau intégré, subset pattern, valeur pré-calculée, duplication volontaire ; le choix dépend du volume et de la lecture.
- **Six collections :** sessions, click_events, logs, customer_feedback, ml_features, inbound_payloads. Schéma flexible (JSON, XML), binaires dans S3, expiration automatique (TTL).
- **Alimentation :** API de collecte, Kafka, DocumentDB ; les données rejoignent ensuite le lakehouse pour les variables froides.
- **Redis :** huit compteurs de fraude (paiements par client, cartes distinctes, clients par carte, montant cumulé, paiements par IP, appareils, paiements par marchand, profil de session) et une liste de blocage d'IP. Structures adaptées (INCR avec TTL, HyperLogLog, Set, Sorted Set, Hash), lecture en moins d'une milliseconde, clés pseudonymisées, reconstruction possible depuis les sources.

Architecture du pipeline de données

Circulation des données entre OLTP, NoSQL, lakehouse, Redis et entraînement du modèle.

- **Deux flux d'entrée :** le flux critique (paiement, via l'outbox) et le flux secondaire (logs, clics, sessions, avis, fraudes confirmées), publié par l'API de collecte dans Kafka sans passer par Aurora.
- **Outbox et Debezium :** l'événement est écrit avec le paiement ; Debezium le publie dans Kafka sans ralentir le paiement. Si Kafka tombe, les événements attendent.
- **Kafka :** livraison « au moins une fois », consommateurs idempotents (MERGE, upsert), file d'erreurs (DLQ).
- **Médaillon :** Bronze (brut, source de rejeu), Silver (typé, validé, sans doublon, quarantaine), Gold (dimensions avant faits, agrégats, variables froides).
- **Fraîcheur :** temps réel pour le paiement et les compteurs, micro-batch de quelques minutes pour Silver, heure ou jour pour Gold, exports de nuit.
- **Boucle de feedback :** les fraudes confirmées par la banque suivent le topic fraud_labels vers Bronze, Silver et Gold, et servent de cible au réentraînement.
- **Orchestration et reprise :** Databricks Workflows (dépendances, relances), cinq alertes CloudWatch ; recalcul depuis Bronze, retour arrière Delta (RESTORE), Redis reconstruit.

Stratégie d'intégration du ML

Comment les modèles s'intègrent à la plateforme : variables, entraînement, déploiement, suivi.

- **Processus en quatre étapes :** extraire les variables, entraîner, déployer, suivre et mettre à jour.
- **Trois cas d'usage :** fraude en temps réel (XGBoost sur ECS Fargate), personnalisation (filtrage collaboratif calculé chaque jour et servi via Redis), analyses prédictives (séries temporelles, résultats rendus à la BI).
- **Variables :** chaudes dans Redis (comportement récent) et froides dans Gold (historique).
- **Entraînement et déploiement :** Databricks et MLflow ; le nouveau modèle (challenger) est comparé à celui en place, validé par le Model Owner, puis étiqueté « production » ; retour arrière possible ; réentraînement chaque semaine.
- **Fraude :** le modèle apprend des vrais résultats renvoyés par la banque ; chaque décision enregistre son score, ses motifs et la version du modèle.
- **Suivi :** qualité du modèle (MLflow, Lakehouse Monitoring), service de scoring (CloudWatch), alerte puis décision du Model Owner.

Plan de sécurité et de conformité

Mesures de protection, de contrôle et de preuve pour RGPD, CCPA et PCI-DSS.

- **Classification :** chaque système a son niveau de sensibilité ; les données critiques sont concentrées dans customer, payment_method et Bronze.
- **Protection des données :** KMS (une clé par domaine, rotation annuelle), TLS partout, Secrets Manager, tokenisation des cartes, pseudonymisation, anonymisation.

- **Droits des personnes :** demandes suivies dans data_subject_request, délai d'un mois ; l'effacement anonymise la personne, les paiements restent conservés (obligation légale) ; sauvegardes et VACUUM bornés dans le temps.
- **Contrôle des accès :** RBAC au moindre privilège, IAM (double authentification, droits temporaires), Unity Catalog (colonnes masquées), revue trimestrielle ; réseau en zones, test et production séparés.
- **Audit :** CloudTrail, audit_log, Unity Catalog et historique Delta ; journaux protégés dans un compte AWS distinct.
- **Surveillance :** alertes CloudWatch, contrôles quotidiens, rapport de conformité, procédure de notification d'une violation sous 72 heures.

Diagramme d'architecture complet

Vue d'ensemble des 20 composants de la plateforme et de leurs liens.

- **Accès et scoring :** AWS WAF, API Gateway, scoring ECS Fargate (XGBoost), Redis pour les variables chaudes.
- **Transactionnel et streaming :** Aurora PostgreSQL Multi-AZ, table outbox, Debezium, Kafka (AWS MSK).
- **Lakehouse :** Bronze, Silver, Gold (Delta Lake sur S3), orchestration Workflows, entraînement MLflow.
- **NoSQL :** AWS DocumentDB et API de collecte.
- **Transverse :** CloudWatch (supervision), KMS (chiffrement), IAM (accès), Unity Catalog (catalogue), CloudTrail (audit).
- **Lecture :** pour chaque composant, une fonction, une technologie retenue, une description et sa sortie vers le composant suivant.

Requêtes SQL et NoSQL

Requêtes répondant aux questions métier clés, sur les trois systèmes.

- **SQL sur l'OLTP :** paiements bloqués sur 24 heures, adresse IP bloquée, carte utilisée par plusieurs clients, clients dépassant trois paiements par heure, demandes d'effacement en retard, preuve d'audit.
- **SQL sur l'OLAP :** chiffre d'affaires net par mois et par pays, trois meilleurs marchands par mois (RANK), produits et catégories (top 10), segmentation par quartiles (NTILE), contrôle de cohérence entre net par transaction et net par produit.
- **Sur le NoSQL :** sessions ressemblant à un bot (ml_features croisé avec sessions) et erreurs techniques derrière les avis négatifs (customer_feedback croisé avec logs).
- **Limite assumée :** les index ne sont pas optimisés.

Présentation de soutenance

Synthèse orale de la conception, avec trois annexes de référence.

- **Déroulé :** le besoin, le stockage, l'architecture, la vélocité, la résilience, la sécurité et conformité, l'intégration de l'IA, puis une synthèse reliant chaque attente du business case à sa réponse.
- **Repères chiffrés :** décision de fraude en 100 ms, lecture des compteurs Redis en moins d'une milliseconde, trois systèmes de données, trois réglementations, 20 briques.
- **Annexe 1 :** les choix techniques et leur justification (stockage, intégrité, performance, fraîcheur, qualité et rejeu, IA, sécurité, exploitation).
- **Annexes 2 et 3 :** architecture complète et tableau des flux (chemin, mode, fraîcheur).

Note sur l'usage de l'intelligence artificielle

Pour rédiger ce document, j'ai utilisé un assistant d'IA générative comme outil d'aide à la rédaction : correction de l'orthographe et de la syntaxe, propositions de structures et de brouillons que j'ai relus et modifiés, comparaison d'options avant décision, vérification de la cohérence d'ensemble. Les choix retenus et la validation finale relèvent de ma responsabilité.